

Лабораторная работа № 1
Основные элементы языка ассемблера
Вариант 7

1. Цель работы

Изучить основные команды ассемблера, работу с регистрами, памятью, символьными данными и консольными функциями ввода-вывода Windows.

2. Постановка задачи

Ввести целое число x в восьмеричной системе и вывести его в десятичной. Вычислить полином

$$y = 11x^2 - 2x + 1$$

и вывести y в двоичной и десятичной системах. Промежуточные данные сохранять в памяти. Не использовать `wsprintfA`; ввод и вывод сопровождать поясняющими сообщениями.

Программа написана на MASM для Win32, модель памяти FLAT, соглашение STDCALL. Ввод допускает цифры 0–7 и необязательный начальный минус. По условию вычисления не должны вызывать переполнение; используемый диапазон $-13972 \leq x \leq 13972$ обеспечивает это для всех промежуточных значений. Некорректный ввод и переполнение выявляются с сообщением об ошибке.

3. Алгоритм

1. Получить дескрипторы консоли через `GetStdHandle`. Вывести приглашение и прочитать строку функцией `ReadConsoleA`. Исключить завершающие CR и LF, учесть знак.
2. Преобразовать восьмеричные цифры слева направо по формуле $v \leftarrow 8v + d$, начиная с $v = 0$; сохранить каждое новое значение в памяти. При необходимости изменить знак и записать число в `x`.
3. Вывести x в десятичной системе. Для преобразования делить модуль числа на 10, сохранять частное и остаток в памяти, записывать цифры в буфер справа налево. Для отрицательного числа добавить минус.
4. Вычислить и сохранить x^2 , $11x^2$, $-2x$, $11x^2 - 2x$ и y в `square`, `quadratic`, `linear`, `partial`, `result` соответственно.
5. Аналогично шагу 3 преобразовать y сначала с основанием 2, затем 10. Вывести пояснения и строки через `WriteConsoleA`. Завершить программу функцией `ExitProcess`.

Цикл вывода выполняется хотя бы один раз, поэтому ноль тоже преобразуется корректно. Буфер заполняется с конца: отдельное обращение строки не требуется.

4. Результаты тестирования

Собранный файл lab1.exe проверен запуском в консоли Windows. Во всех десяти корректных тестах фактические значения совпали с ожидаемыми; код завершения равен 0. Вход в таблице записан в системе счисления 8.

Ввод	x_{10}	y_2	y_{10}
0	0	1	1
1	1	1010	10
-1	-1	1110	14
7	7	1000001110	526
10	8	1010110001	689
-10	-8	1011010001	721
123	83	10010011101011110	75614
00017	15	100110001110	2446
33224	13972	111111111111100001000000001001	2147356681
-33224	-13972	111111111111101110101001011001	2147412569

Дополнительно проверены вводы 8, -, 12a и 33225: программа сообщает об ошибке и возвращает код 1. Первые три строки нарушают формат, последняя вызывает переполнение.

Например, для $x = 10_8 = 8_{10}$ промежуточные значения равны 64, 704, -16, 688; результат $y = 689_{10} = 1010110001_2$. Вывод программы:

```
Enter x in base 8 (digits 0-7, optional minus): 10
x (decimal) = 8
11*x*x - 2*x + 1 (binary) = 1010110001
11*x*x - 2*x + 1 (decimal) = 689
```

5. Текст программы

```
1 .386
2 .MODEL FLAT, STDCALL
3 .STACK 4096
4
5 EXTERN GetStdHandle@4:PROC
6 EXTERN ReadConsoleA@20:PROC
7 EXTERN WriteConsoleA@20:PROC
8 EXTERN ExitProcess@4:PROC
9
10 .DATA
11 prompt      DB 'Enter x in base 8 (digits 0-7, optional minus): ',0
12 xLabel      DB 'x (decimal) = ',0
13 binLabel    DB '11*x*x - 2*x + 1 (binary) = ',0
14 decLabel    DB '11*x*x - 2*x + 1 (decimal) = ',0
15 errorText   DB 'Error: invalid octal input or 32-bit overflow.',13,10,0
16 newLine     DB 13,10,0
17 inputHandle DD ?
18 outputHandle DD ?
19 readCount   DD ?
20 written     DD ?
21 inputBuffer DB 64 DUP (?)
22 outputBuffer DB 34 DUP (?)
23 textAddress DD ?
24 textLength  DD ?
25 position    DD ?
26 negative    DD ?
27 digit       DD ?
28 value       DD ?
29 x           DD ?
```

```

30 square      DD ?
31 quadratic   DD ?
32 linear      DD ?
33 partial     DD ?
34 result      DD ?
35 number      DD ?
36 base        DD ?
37 quotient    DD ?
38 remainder   DD ?
39
40 .CODE
41 ; EAX points to a zero-terminated string.
42 PrintText PROC
43     MOV textAddress, EAX
44     MOV textLength, 0
45 countChars:
46     MOV EDX, textLength
47     CMP BYTE PTR [EAX+EDX], 0
48     JE writeText
49     INC textLength
50     JMP countChars
51 writeText:
52     PUSH 0
53     PUSH OFFSET written
54     PUSH textLength
55     PUSH textAddress
56     PUSH outputHandle
57     CALL WriteConsoleA@20
58     RET
59 PrintText ENDP
60
61 ; Read and convert a signed octal integer. Result is stored in x.
62 ReadOctal PROC
63     PUSH 0
64     PUSH OFFSET readCount
65     PUSH 64
66     PUSH OFFSET inputBuffer
67     PUSH inputHandle
68     CALL ReadConsoleA@20
69     TEST EAX, EAX
70     JZ inputError
71     CMP readCount, 2
72     JB inputError
73     SUB readCount, 2
74     MOV EDX, readCount
75     CMP inputBuffer[EDX], 13
76     JNE inputError
77     CMP inputBuffer[EDX+1], 10
78     JNE inputError
79     MOV position, 0
80     MOV negative, 0
81     MOV value, 0
82     CMP inputBuffer[0], '-'
83     JNE checkDigits
84     MOV negative, 1
85     INC position
86 checkDigits:
87     MOV EDX, position
88     CMP EDX, readCount
89     JAE inputError
90 readDigit:
91     MOV EDX, position
92     MOVZX EAX, inputBuffer[EDX]
93     CMP EAX, '0'
94     JB inputError
95     CMP EAX, '7'
96     JA inputError
97     SUB EAX, '0'
98     MOV digit, EAX

```

```

99     MOV EAX, value
100    IMUL EAX, 8
101    JO inputError
102    MOV value, EAX
103    ADD EAX, digit
104    JO inputError
105    MOV value, EAX
106    INC position
107    MOV EDX, position
108    CMP EDX, readCount
109    JB readDigit
110    CMP negative, 0
111    JE storeX
112    NEG value
113 storeX:
114     MOV EAX, value
115     MOV x, EAX
116     RET
117 ReadOctal ENDP
118
119 ; Convert number to base 2 or 10, filling the buffer from right to left.
120 PrintNumber PROC
121     MOV position, 33
122     MOV outputBuffer[33], 0
123     MOV negative, 0
124     MOV EAX, number
125     CMP EAX, 0
126     JGE magnitudeReady
127     MOV negative, 1
128     NEG EAX
129 magnitudeReady:
130     MOV quotient, EAX
131 nextDigit:
132     MOV EAX, quotient
133     XOR EDX, EDX
134     DIV base
135     MOV quotient, EAX
136     MOV remainder, EDX
137     ADD EDX, '0'
138     DEC position
139     MOV ECX, position
140     MOV outputBuffer[ECX], DL
141     CMP quotient, 0
142     JNE nextDigit
143     CMP negative, 0
144     JE printDigits
145     DEC position
146     MOV ECX, position
147     MOV outputBuffer[ECX], '-'
148 printDigits:
149     MOV EAX, OFFSET outputBuffer
150     ADD EAX, position
151     CALL PrintText
152     MOV EAX, OFFSET newLine
153     CALL PrintText
154     RET
155 PrintNumber ENDP
156
157 MAIN PROC
158     PUSH -10
159     CALL GetStdHandle@4
160     MOV inputHandle, EAX
161     PUSH -11
162     CALL GetStdHandle@4
163     MOV outputHandle, EAX
164     MOV EAX, OFFSET prompt
165     CALL PrintText
166     CALL ReadOctal
167

```

```

168     MOV EAX, OFFSET xLabel
169     CALL PrintText
170     MOV EAX, x
171     MOV number, EAX
172     MOV base, 10
173     CALL PrintNumber
174
175     ; Store every intermediate value of  $11*x*x - 2*x + 1$ .
176     MOV EAX, x
177     IMUL EAX, x
178     JO inputError
179     MOV square, EAX
180     IMUL EAX, 11
181     JO inputError
182     MOV quadratic, EAX
183     MOV EAX, x
184     IMUL EAX, -2
185     JO inputError
186     MOV linear, EAX
187     ADD EAX, quadratic
188     JO inputError
189     MOV partial, EAX
190     ADD EAX, 1
191     JO inputError
192     MOV result, EAX
193
194     MOV EAX, OFFSET binLabel
195     CALL PrintText
196     MOV EAX, result
197     MOV number, EAX
198     MOV base, 2
199     CALL PrintNumber
200     MOV EAX, OFFSET decLabel
201     CALL PrintText
202     MOV EAX, result
203     MOV number, EAX
204     MOV base, 10
205     CALL PrintNumber
206     PUSH 0
207     CALL ExitProcess@4
208 MAIN ENDP
209
210 inputError:
211     MOV EAX, OFFSET errorText
212     CALL PrintText
213     PUSH 1
214     CALL ExitProcess@4
215 END MAIN

```

6. Вывод

Реализованы ввод восьмеричного числа, вычисление полинома и ручное преобразование чисел для вывода. Проверены ноль, оба знака входного числа, ведущие нули, границы диапазона и обработка ошибок. Результаты тестов подтверждают правильность работы программы.